# Secure Coding

## Java Advanced Developer

StayAhead Training; Dec, 2025

# About Secure Coding

## Secure Coding

- Java and the JVM have features that mitigate mistakes and attacks:
  - The language is **type-safe**
  - The runtime **manages memory**
  - The runtime **performs bounds-checking** on arrays
- But it is necessary to avoid bugs that may weaken security

# Fundamentals

## Secure Coding

- Prefer to have obviously no flaws rather than no obvious flaws
- Design APIs with security in mind
- Avoid duplication - code & data is not treated consistently when duplicated
- Restrict privileges - decompose applications; use separate VMs
- Establish trust boundaries - sanitise & validate data; beware third-party libs

# Fundamentals

## Secure Coding

- Minimise the number of security checks
- Encapsulate - private fields; succinct and coherent interfaces
- Document security-related info., e.g. required permissions & exceptions
- Secure third-party code - apply updates in a timely manner

# Denial of Service

## Secure Coding

- Beware of activities that may use resources disproportionately
- Release resources in all cases - use `try` with resources
- Resource limit checks should not suffer from integer overflow
- Implement robust error/exception handling for services
- Avoid using user input as hash keys - see hash collision attack

# Confidential Information

## Secure Coding

- Purge sensitive information from exceptions
- Do not log sensitive information
- Consider purging sensitive information from memory after use

# Injection and Inclusion

## Secure Coding

- Generate valid formatting - e.g. use well-tested libs for creating XML
- Avoid dynamic SQL - use `PreparedStatement` instead of `Statement`
- XML and HTML generation requires care - see XSS attack
- Avoid untrusted data on the command line - encode args or use a file
- Restrict XML inclusion

# Injection and Inclusion

## Secure Coding

- Take care with BMP files - they may contain references to local files
- Disable HTML display in Swing components - some interpret text as HTML
- Take care interpreting untrusted code - run it in a secure sandbox
- Prevent injection of exceptional floating point numbers, e.g. NaN

# Accessibility and Extensibility

## Secure Coding

- Limit the accessibility of classes, interfaces, methods, and fields
- Use modules to hide internal packages
- Limit the exposure of `ClassLoader` instances
- Limit the extensibility of classes and methods - see `final` and `sealed`
- Understand how a superclass can affect subclass behaviour

# Input Validation

## Secure Coding

- Validate inputs - beware malicious inputs
- Validate output from untrusted objects as input
- Define wrappers around native methods
- Verify API behaviour related to input validation

# Mutability

## Secure Coding

- Prefer immutability for value types
- Create copies of mutable output values - see `Object.clone`
- Create safe copies of mutable and subclassable input values
- Support copy functionality for a mutable class
- Do not trust identity equality when overridable on input reference objects - `Object.equals` is overridable

# Mutability

## Secure Coding

- Treat passing input to untrusted object as output
- Treat output from untrusted object as input
- Define wrapper methods around modifiable internal state
- Make `public static` fields `final`
- Ensure `public static final` field values are constants (immutable)

# Mutability

## Secure Coding

- Do not expose mutable statics
- Do not expose modifiable collections - public fields and getters

# Object Construction

## Secure Coding

- Avoid exposing constructors of sensitive classes - use `static` factories
- Defend against partially initialised instances of non-final classes
- Prevent constructors from calling methods that can be overridden
- Defend against cloning of non-final classes

# Serialisation and Deserialisation

## Secure Coding

- Avoid serialisation for security-sensitive classes - makes fields `public`
- Guard sensitive data during serialisation - see `transient`
- View deserialisation the same as object construction
- Duplicate security-related checks during serialisation and deserialisation
- Filter untrusted serial data - see Serialisation Filtering since v9

# Access Control

## Secure Coding

- The Security Manager controls how code interacts with external resources
- It is enabled on startup, e.g. `$ java -Djava.security.manager ...`
- It dictates whether sensitive operations may be executed
- It was deprecated in v17 and permanently disabled in v24 because